

Exercises on NumPy

Exercise 1: Create a 4x4 array containing integers from 10 to 25. Convert this array into a complex128 data type. Then, add an imaginary part of 1j to all elements on the main diagonal and 2j to all other elements using vectorized operations.

Exercise 2: Generate a 1D array of 36 sequential integers. Reshape it into a 3D array with the shape (3, 3, 4). Using the `.transpose()` method (not just `.T`), swap the first and third axes so the final shape becomes (4, 3, 3).

Exercise 3: Create a 10x10 matrix of zeros. Using **only two slicing statements** (no loops or if statements), create a "border" of 1s around the edges and a "cross" of 5s in the middle (the 5th and 6th rows and columns).

Exercise 4: Generate a 6x6 array of random integers between -50 and 50. Create a boolean mask to identify elements that are both **negative** and **even**. Use this mask to replace those specific elements with their absolute value squared ($|x|^2$).

Exercise 5: The Sigmoid function is defined as: $f(x) = \frac{1}{1+e^{-x}}$.

Create a 1D array `x` with 20 points linearly spaced between -5 and 5. Calculate the Sigmoid values for the entire array using only `np.exp` and basic arithmetic in a single vectorized line.

Exercise 6: Create an 8x8 matrix representing a chessboard. Use slicing with steps to fill it with 0s and 1s in a checkerboard pattern (0 for white squares, 1 for black). Do this in exactly two lines of code without using `np.tile`.

Exercise 7: Create two 5x5 arrays, A (random floats between 0-1) and B (random floats between 0-1). Create a third array C where the element is taken from A if $A > B$, and from B otherwise. Perform this using a single NumPy function mentioned in the "Logic" section of the notes.

Exercise 8: Generate a 1D array of 100 random values from a normal distribution. Find the **mean** and **standard deviation**. Create a new array that contains only the values that fall within **one standard deviation** of the mean (i.e., $|x - \mu| < \sigma$).

Exercise 9: In machine learning, we often scale data between 0 and 1. Given a 2D array of "sensor data" (shape 5x3) with random values:

1. Find the minimum and maximum values of the entire array.
2. Apply the formula $x_{\text{scaled}} = \frac{x - \min}{\max - \min}$ to the entire array using vectorization.

Exercise 10: Given a 1D array representing a "point" in 3D space: `p = [3, 4, 5]` and a 2D array points of shape (10, 3) representing 10 different points. Calculate the **Euclidean distance** from `p` to every point in points simultaneously.

Hint: Use `(points - p)**2`, `np.sum` with the axis parameter, and `np.sqrt`